学生公寓管理系统

Android 客户端 + Spring Boot 服务端 · 双端设计文档

本文档描述学生公寓管理系统的双端设计与关键实现决策：客户端负责交互与本地缓存，服务端承担业务规则与数据权威。设计围绕三条主线展开——数据可信（服务端为唯一权威）、错误可定位（统一响应与分层错误码）、结构可演进（两端各自分层，数据访问不耦合于界面）。

文档类型	系统设计说明
项目性质	课程设计延伸 · 独立开发
客户端	Android · Java 8 · Room · Retrofit
服务端	Spring Boot 3.2 · MyBatis · MySQL 8
代码规模	客户端 71 个类 / 约 7200 行 · 服务端 16 个类 / 约 900 行
版本	v1.0

目　录

1　项目概述 5

1.1　设计目标 5

1.2　功能范围 5

2　系统架构 7

2.1　客户端分层 7

2.2　服务端分层 7

2.3　一次请求的完整链路 8

3　数据设计 9

3.1　表结构 9

3.2　两处关键字段设计 9

3.3　表间关系 10

4　接口设计 11

4.1　接口清单 11

4.2　统一响应结构 11

4.3　错误码分层 11

4.4　响应样例（真实抓取） 12

4.5　分页语义 12

5　关键实现与设计决策 14

决策一：数据访问收口到 DAO 14

决策二：手写 DAO 迁移到 Room 14

决策三：服务端为唯一权威数据源 15

决策四：数据库版本迁移 16

决策五：统一响应与异常语义 17

6 界面展示 18

7 质量保障 23

7.1 自动化测试 23

7.2 接口验证记录 23

7.3 已修复的典型缺陷 24

8 技术栈与工程实践 25

8.1 技术栈 25

8.2 工程实践 25

9 范围与约束 26

1　项目概述

学生公寓管理系统面向高校宿舍场景，为「学生」与「管理员」两类角色提供日常事务处理的统一入口：请假审批、报修、进出打卡、卫生评分、缴费、公告与访客登记。

系统采用客户端 + 服务端的双端结构：Android 客户端负责交互与本地缓存，Spring Boot 服务端承担业务规则与数据权威。两端通过 REST 接口协作，职责边界清晰。

1.1　设计目标

目标	落实方式
数据可信	服务端为唯一权威数据源，客户端数据仅作缓存，冲突时以服务端为准
错误可定位	统一响应结构 + 分层错误码，调用方可直接判断失败发生在哪一层
结构可演进	客户端与服务端各自分层，数据访问与业务规则不耦合于界面
离线可查看	客户端保留本地缓存，网络不可用时降级展示并明确标注数据来源

1.2　功能范围

角色	功能
学生端	请假上报与审批状态、报修提交、进出打卡、卫生评分查询、缴费与欠费查询、公告查看

角色	功能
管理端	请假审批（同意/驳回并填写原因）、报修处理、进出记录、卫生评分录入、缴费登记、公告发布、学生信息管理、访客登记

2 系统架构

两端各自分层，层与层之间只通过明确的接口通信。这样做的直接收益是：改界面不动数据逻辑，改数据来源不动界面。

2.1 客户端分层

层	职责	约束
UI（Activity / Fragment）	展示与交互、收集输入	不出现 SQL、不出现 HTTP 调用
Repository	决定数据从哪来：网络优先、失败回退缓存	唯一的业务数据入口
DAO / ApiService	本地读写（Room）、远程调用（Retrofit）	只做存取，不含业务判断
存储 / 网络	SQLite、HTTP	由上层驱动，不反向依赖

2.2 服务端分层

层	职责	约束
Controller	接收参数、调用 Service、包装统一响应	不写业务规则
Service	业务校验、编排、事务边界	不出现 HTTP 对象、不写 SQL
Mapper	SQL 与结果映射	不含业务判断
MySQL	数据权威存储	唯一事实来源

2.3 一次请求的完整链路

```
Activity
  → Repository            决定数据来源（网络 / 缓存）
    → Retrofit → OkHttp → HTTP
      → Tomcat
        → Controller       接参数、包装响应
          → Service        业务校验、事务
            → Mapper       执行 SQL
              → MySQL      权威数据
  ← 结果按原路返回，成功后回写本地缓存
```

这条链路上每一段都可能失败，且失败形态完全不同。因此客户端把错误分为三层处理：传输层（连不上）、HTTP 层（被拒绝）、业务层（业务规则不允许），并用不同的话术提示用户。

3 数据设计

服务端以 MySQL 存储业务数据，共 9 张表；客户端 Room 保留同名结构用于本地记录，并额外维护一张缓存表。

3.1 表结构

表	职责	关键字段
user	账号与个人信息（0 学生 / 1 管理员）	account、userType、room
repairs	报修单	userId、status、orderDate、date
temperature	请假单（沿用旧表名）	userId、status、rejectReason
come	进出打卡	userId、typeId、status
score	寝室卫生评分	room、desktop、bed、balcony
task	公告	title、content
pay	缴费记录	dorNum、money
fang	访客登记	name、dorNum、reason
room_type	房型与年费（欠费计算依据）	capacity、price
cached_student	客户端学生列表缓存（非权威）	id、cachedAt

3.2 两处关键字段设计

设计	说明

设计	说明
orderDate 与 date 分离	orderDate 是学生提交时间，由服务端写入；date 是管理员确认处理的时间，未处理时为空串。两个时间点语义不同，不能合并
status 用整型枚举	0 未修 / 1 已修（报修），0 待审核 / 1 已同意 / 2 已驳回（请假）。状态流转只能由服务端决定，客户端不参与

3.3 表间关系

repairs、temperature、come 通过 userId 关联 user，列表查询使用内连接带出学生姓名与寝室；score、task、pay、fang 为不依赖具体用户的独立记录。联表所需的展示字段由服务端 join 后一次性返回，客户端无需二次查询。

4 接口设计

4.1 接口清单

方法	路径	说明
GET	/students?page=&size=	学生列表（分页）
POST	/repairs	新增报修（参数校验，仅学生可提交）
GET	/repairs?status=	按状态筛选报修（支持中文与 0/1）

4.2 统一响应结构

```
{
  "code": 0,          // 0 成功；非 0 为业务错误码
  "message": "ok",
  "data": { }
}
```

HTTP 状态码表达「请求本身是否成立」，业务 code 表达「业务是否成立」。两者分层，网关与监控看前者，业务代码看后者。

4.3 错误码分层

HTTP	场景	由谁决定
400	参数校验失败 / 业务规则不满足	校验注解、Service
404	路径或资源不存在	框架、Service
405	请求方法不支持	框架

HTTP	场景	由谁决定
503	依赖不可用（数据库连不上）	全局异常处理器
500	未预期异常（兜底）	全局异常处理器

4.4 响应样例（真实抓取）

```
GET /students?page=1&size=3
{"code":0,"message":"ok","data":{"total":6,"page":1,"size":3,"pages":2,
  "list":[{"id":7,"account":"stu006","name":"孙八",
           "sex":"女","phone":"13800000006","room":"308"}]}}

POST /repairs  {"userId":2,"thing":"台灯","cause":"不亮了"}
{"code":0,"message":"ok","data":{"id":4,"userId":2,"thing":"台灯",
  "cause":"不亮了","status":0,"statusText":"未修",
  "orderDate":"2026-11-02 19:25:22","date":"",
  "studentName":"张三","studentRoom":"103"}}
```

第二条响应中的 studentName 与 studentRoom 不属于 repairs 表，是服务端 join user 后直接返回的——这正是把联表放在服务端带来的收益。

4.5 分页语义

字段	含义
total	总条数，翻页时不变
page / size	当前页码（从 1 开始）、每页条数
pages	总页数 = ceil(total / size)
offset	服务端计算：(page - 1) × size，且 page、size 均有

字段	含义
	上下限保护

5　关键实现与设计决策

以下五项是本项目里真正影响结构的决定，每项记录背景、选择与代价。

决策一：数据访问收口到 DAO

背景	**原实现中每个页面自己开库、自己拼 SQL、自己按列位置取值，同一段映射在 5 个文件里各写一遍；全项目有 89 处按列下标取值**
选择	抽出 8 个 DAO 类，外部只接触实体对象；列取值改为按列名获取
代价	多一层间接；简单查询也要写 DAO 方法
结论	字段顺序变化不再造成静默错数据，重复映射只此一份

决策二：手写 DAO 迁移到 Room

背景	**DAO 分层解决了 SQL 散落，但 SQL 写错的代价仍在运行时——表名、列名拼错要到设备上才暴露**
选择	8 个 DAO 改为 Room 注解，SQL 由注解处理器在编译期校验
代价	引入注解处理器；受 Room 版本与 compileSdk 约束（本项目只能用 2.4.x）
结论	SQL 错误提前到编译期；联表字段用独立 POJO + @Embedded 描述，实体不再「有时有值有时没有」

决策三：服务端为唯一权威数据源

背景	**客户端有本地库，服务端也有库。若两者都参与判断，就必须信任客户端，而客户端无法被信任（可反编译、可改本地库、可改请求体）**
选择	服务端为唯一权威：写入、状态流转、时间戳由服务端决定；客户端数据仅作缓存，冲突时以服务端为准
代价	离线只能查看，不能在本地裁决业务状态
结论	客户端只提交「意图」，服务端决定「结果」；这一条同时构成防注入与防本地篡改的基础

落地时读与写采用不同策略，这个不对称是刻意的：

操作	**策略**	**原因**
读（学生列表）	网络优先 → 成功则整体覆盖本地缓存 → 失败才回退缓存，并标注数据来源	离线也能查看；但必须让用户知道看到的是缓存
写（提交报修）	只走服务端，不回退	写操作降级会让失败看起来像成功，属于最危险的一类缺陷

```
StudentRepository.loadStudents()
  1. 请求服务端（权威）
  2. 成功 → 结果整体覆盖 cached_student → 交付，来源 = 服务端
  3. HTTP 失败 / 业务失败 / 连不上
       → 读缓存：有则交付并标注来源 = 本地缓存，无则报错
```

决策四：数据库版本迁移

背景	**新增缓存表需要把本地库从 v8 升到 v9。此前的 v7 → v8 只能使用破坏性迁移——旧库由 SQLiteOpenHelper 手写建立，没有 Room 的 identity hash，无法写忠实迁移**
选择	v8 → v9 编写精确 Migration：只新增 cached_student 一张表，不触碰既有表与数据
代价	迁移语句必须与 Room 生成的建表语句逐字一致，否则启动即校验失败
结论	保留数据完成升级；校验方式是提取 Room 生成的建表语句与迁移语句比对，两边均为 140 字符且完全一致

```
static final Migration MIGRATION_8_9 = new Migration(8, 9) {
    public void migrate(SupportSQLiteDatabase db) {
        db.execSQL("CREATE TABLE IF NOT EXISTS `cached_student` (...)");
        // 只加表，不动其它表的数据
    }
};
```

决策五：统一响应与异常语义

背景	**若各类异常都落到兜底处理器，调用方只会看到「500 服务器内部错误」，无法判断是参数问题、路径问题还是依赖不可用**
选择	按语义分别处理：参数 400、路径 404、方法 405、依赖不可用 503；仅未预期异常返回 500 并记录日志
代价	每类异常都要显式声明，漏掉一类就会被兜底吞掉
结论	错误可被直接定位；实测曾因兜底处理器吞掉 NoResourceFoundException，使 404 变成 500，后单独补齐处理器

6　界面展示



登录页 · 学生 / 管理员统一入口



管理员首页 · 7 个管理模块



请假审批 · 待审核 / 已同意 / 已驳回



驳回 · 必须填写驳回原因



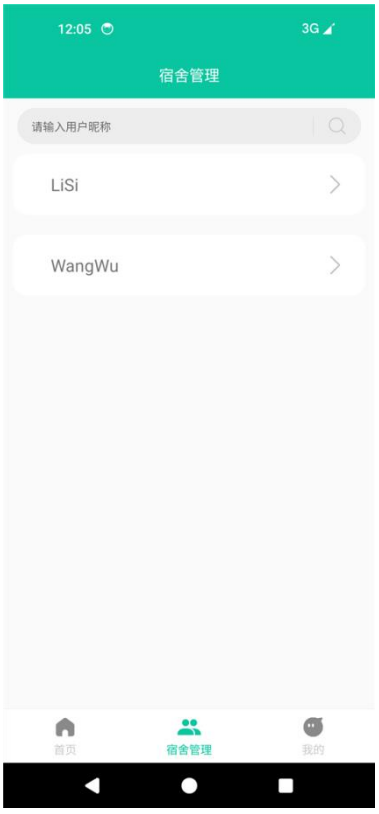
报修处理 · 一键标记已修



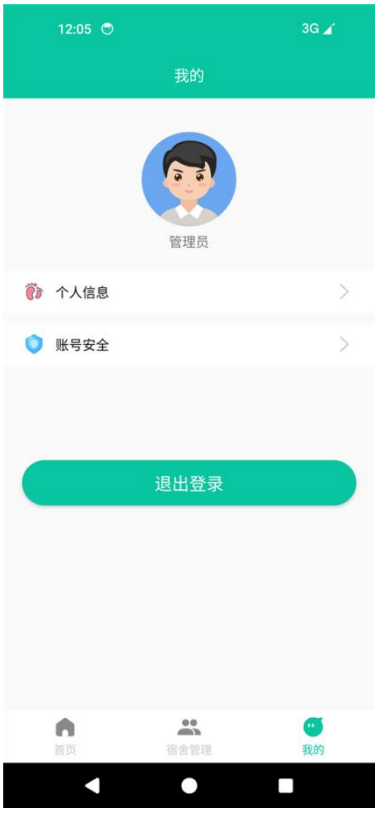
卫生评分 · 按寝室录入



公告 · 发布与查看



宿舍管理 · 学生增删改查



管理员 · 我的



管理员独立登录入口



学生首页 · 7 个功能入口



请假上报 · 表单 + 本人记录与审批状态



欠费查询 · 年费从 room_type 表读取



扫码支付 · 收款码



公告列表



评分列表



出入打卡



学生 · 我的



缴费管理 · 登记欠费 + 查看记录

7　质量保障

7.1　自动化测试

客户端 3 个 JVM 单元测试，无需模拟器即可运行：以真实服务端返回的 JSON 验证 DTO 字段映射。Gson 按字段名反射填充，字段名错一个字母即得到 null 且不报错，因此这组测试守的是「客户端 DTO 与服务端格式」的契约。

```
NetMappingTest
  OK studentsResponse_mapsEveryField
  OK repairResponse_includesJoinedUserFields
  OK errorResponse_mapsCodeAndMessage

BUILD SUCCESSFUL - 3 tests, 0 failures
```

7.2　接口验证记录

用例	请求	结果
分页首屏	GET /students?page=1&size=3	200，total=6、pages=2，返回 3 条
分页边界	GET /students?page=0&size=0	200，参数自动兜底为 page=1、size=10
新增报修	POST /repairs	200，主键回填 id=4，联表字段正确
条件筛选	GET /repairs?status=待处理	200，返回 3 条未修记录
参数校验	POST /repairs 且 thing 为空	400，返回字段级错误
非法状态	GET /repairs?status=abc	400，明确提示合法取值

用例	请求	结果
路径不存在	GET /nope	404
方法不支持	POST /students	405
数据库不可用	停掉 MySQL 后请求	503，并提示确认数据库状态

7.3 已修复的典型缺陷

现象	根因	处理
宿舍管理列表恒为空，且不报错	建表时列未声明类型，SQLite 按 NONE 亲和性不做转换，文本与整型比较不相等	补齐列类型并重建表
删除卫生评分却删掉了公告	删除语句的目标表写错	修正目标表
公告标题含百分号即崩溃	把用户数据当成格式化字符串	改为占位符格式化
管理员每次启动需重新登录	以 userId > 0 判断登录态，而管理员 id 恰为 0	引入独立登录标记
Android 5.x 上进入页面即崩溃	使用已废弃 Fragment，其回调在低版本不存在	迁移到 AndroidX Fragment
数据库不可用时只报 500	异常被兜底处理器吞掉	按语义新增 503 处理器，并把原因写入响应体

8 技术栈与工程实践

8.1 技术栈

层	技术选型
客户端	Java 8、Room 2.4.3、Retrofit 2.9、OkHttp 4.12、RecyclerView、ViewBinding、Glide
服务端	Spring Boot 3.2、MyBatis 3.0、MySQL 8.0、H2（本地演示）
构建与工具	Gradle 8.9、AGP 8.7.3、JDK 17、MySQL 命令行、curl

8.2 工程实践

实践	说明
文档化	16 篇源码学习笔记与接手说明，覆盖登录链路、数据层演进、网络请求链路等
可重复构建	客户端与服务端均提供一键启动脚本；本文档由脚本从结构化内容生成，可随时重建
参数化 SQL	全部 SQL 使用预编译占位符，无字符串拼接
日志可观测	服务端按包开启 MyBatis SQL 日志，客户端记录完整请求与响应

9 范围与约束

以下为项目当前明确的范围边界，属课题范围内的主动取舍：

项	范围
鉴权	本期不包含登录态与令牌体系，接口在可信内网环境调用
密码存储	沿用教学项目的明文存储，未引入加盐哈希
数据源收敛	Repository 已覆盖新增联调页；存量页面仍直读本地库，按其承载的功能分别演进
测试范围	已覆盖网络层数据契约；UI 与数据库行为以手工验证记录为准
运行环境	单体部署，未涉及集群、缓存中间件与高并发场景

这些边界均不影响系统作为完整双端设计的自洽性：客户端与服务端职责清晰，数据权威唯一，错误语义完整，关键决策有据可查。